Listing A.2 Original main.tsx file using JSX

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.tsx'
createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Listing A.3 Transpiled main.tsx (from JSX to JavaScript)

```
import __vite__cjsImport0_react_jsxRuntime from "/node_modules/.v
const jsx = __vite__cjsImport0_react_jsxRuntime["jsx"];
import __vite__cjsImport1_react from "/node_modules/.vite/deps/re
const StrictMode = __vite__cjsImport1_react["StrictMode"];
import __vite__cjsImport2_reactDom_client from "/node_modules/.vi
const createRoot = __vite__cjsImport2_reactDom_client["createRoot
import "/src/index.css";
import App from "/src/App.tsx";
createRoot(document.getElementById("root")).render(/* @__PURE__ *
jsx(StrictMode, {
    children: /* @__PURE__ */
    jsx(App, {})
}));
```

This comparison demonstrates why JSX is preferred: its concise, intuitive syntax significantly improves developer productivity and code maintainability compared to the verbose transpiled output.

A.3.2 Best Practices for JSX

Table A.4 outlines best practices for breaking down complex UIs, using clear component names, and keeping JSX logic concise and maintainable.

Table A.4 Component structure and organization

Recommendation	Example
Break down complex UI into	`<Header />`, `<Footer />`,